

QUANTIFYING MEMORIZATION ACROSS NEURAL LANGUAGE MODELS

Nicholas Carlini^{*}
Katherine Lee^{1,3}

Daphne Ippolito^{1,2}
Florian Tramèr¹

Matthew Jagielski¹
Chiyuan Zhang¹

¹Google Research

²University of Pennsylvania

³Cornell University

ABSTRACT

Large language models (LMs) have been shown to memorize parts of their training data, and when prompted appropriately, they will emit the memorized training data verbatim. This is undesirable because memorization violates privacy (exposing user data), degrades utility (repeated easy-to-memorize text is often low quality), and hurts fairness (some texts are memorized over others).

We describe three log-linear relationships that quantify the degree to which LMs emit memorized training data. Memorization significantly grows as we increase (1) the capacity of a model, (2) the number of times an example has been duplicated, and (3) the number of tokens of context used to prompt the model. Surprisingly, we find the situation becomes more complicated when generalizing these results across model families. On the whole, we find that memorization in LMs is more prevalent than previously believed and will likely get worse as models continue to scale, at least without active mitigations.

1 INTRODUCTION

The performance of neural language models has continuously improved as these models have grown from millions to trillions of parameters (Fedus et al., 2021), with their training sets similarly growing from millions to trillions of tokens. In anticipation of future, even larger models trained on minimally curated datasets, it is important to quantify factors that lead to increased memorization of a model’s training set. Indeed, recent work has shown that *training data extraction attacks* are a practical threat for current language models (Carlini et al., 2020); an adversary interacting with a pretrained model can extract individual sequences that were used to train the model.

While current attacks are effective, they only represent a lower bound on how much memorization occurs in existing models. For example, by querying the GPT-2 language model, Carlini et al. (2020) (manually) identified just 600 memorized training examples out of a 40GB training dataset. This attack establishes a (loose) lower bound that at least 0.00000015% of the dataset is memorized. In contrast, we are able to show that the 6 billion parameter GPT-J model (Black et al., 2021; Wang and Komatsuzaki, 2021) **memorizes at least 1% of its training dataset**: The Pile (Gao et al., 2020).

In addition to prior work’s loose estimates of models’ memorization capabilities, there is a limited understanding of how memorization varies across different neural language models and datasets of different scales. Prior studies of memorization in language models either focus on models or datasets of a fixed size (Carlini et al., 2019; Zhang et al., 2021; Thakkar et al., 2020) or identify a narrow memorization-versus-scale relationship (Carlini et al., 2020; Lee et al., 2021). While McCoy et al. (2021) broadly study the extent to which language models memorize, their focus is on how to avoid the problem and ensure novelty of model outputs, rather than on studying model risk through identifying the maximal amount of data memorization.

^{*}Authors ordered alphabetically.

This paper addresses both of the above open questions by comprehensively quantifying memorization across three families of neural language models and their associated datasets. We leverage access to each model’s original training set to provide order-of-magnitude more precise bounds on the amount of extractable data that an adversary could recover than in prior works.

We first construct a set of prompts from the model’s training set. By feeding prefixes of these prompts into the trained model, we check whether the model has the ability to complete the rest of the example verbatim. This allows us to measure memorization across models, datasets, and prompts of varying sizes. We identify three properties that significantly impact memorization:

1. **Model scale:** Within a model family, larger models memorize $2\text{--}5\times$ more than smaller models.
2. **Data duplication:** Examples repeated more often are more likely to be extractable.
3. **Context:** It is orders of magnitude easier to extract sequences when given a longer context.

Our analysis suggests that future research on neural language modeling will need to take steps to prevent future (larger) models from memorizing their training datasets.

2 RELATED WORK

There is extensive prior work that qualitatively studies memorization in neural language models. Prior work has demonstrated *extraction attacks* that recover memorized data including URLs, phone numbers, and other personal information (Carlini et al., 2020; Ziegler, 2021)—or synthetically injected “canaries” (Carlini et al., 2019; Henderson et al., 2018; Thakkar et al., 2020; Thomas et al., 2020). However most of these works are qualitative and aim to demonstrate the existence of extractable data, rather than precisely quantifying how much models memorize. For example, the unprompted memorization evaluation of Carlini et al. (2020) found just 600 examples of memorization in GPT-2. Our paper aims to establish tighter bounds on the fraction of a dataset that is memorized.

Our analysis is relevant to the broad literature on privacy attacks on machine learning. For example, membership inference attacks (Shokri et al., 2017; Yeom et al., 2018) let an adversary detect the presence of a given example in a model’s training set; other forms of data leakage let an adversary learn dataset properties (Ganju et al., 2018; Fredrikson et al., 2015). We focus on extraction attacks due to their relevance for language modeling—extraction implies significant leakage from a model, and grows with data duplication (Lee et al., 2021), a common feature of large-scale text datasets.

Various definitions of memorization in deep neural networks have been studied in prior work (Carlini et al., 2019; 2020; Feldman and Zhang, 2020; Zhang et al., 2021). A detailed comparison with those existing formulations is presented in Section 3.1. One leading general memorization definition is differential privacy (Dwork et al., 2006), which formalizes the idea that removing any one example from the training set should not change the trained model. However, while differential privacy protects a single user’s private information, it is ineffective for preventing memorization of highly duplicated data, and does not capture the complexity of social, linguistic data (Brown et al., 2022). Also, differentially private learning algorithms (Abadi et al., 2016) generally suffer from expensive computation, slow convergence, and poor model utility, despite recent advances (Anil et al., 2021).

In concurrent work, Kandpal et al. (2022) study how often models emit memorized data as a function of data duplication. Their analysis focuses on evaluating why training data extraction attacks succeed. In contrast, we explicitly prompt models with training data prefixes in order to measure memorization in the worst case, something that a practical attack cannot necessarily do.

Prior scaling hypotheses. Our motivation to study scaling phenomena stems from anecdotal evidence in prior work that memorization ability relates to various aspects of scale. In particular, our analysis on model scale is informed by preliminary experiments in (Zhang et al., 2017; Carlini et al., 2020), our data duplication experiments follow in the line of Lee et al. (2021), and our context length experiments build on hypotheses by Carlini et al. (2020); Ziegler (2021).

3 METHODOLOGY

3.1 DEFINITION OF MEMORIZATION

To begin, we first select a precise definition for memorization:

Definition 3.1. A string s is *extractable with k tokens of context* from a model f if there exists a (length- k) string p , such that the concatenation $[p \parallel s]$ is contained in the training data for f , and f produces s when prompted with p using greedy decoding.

For example, if a model’s training dataset contains the sequence “My phone number is 555-6789”, and given the length $k = 4$ prefix “My phone number is”, the most likely output is “555-6789”, then this sequence is extractable (with 4 words of context). We focus on greedy sampling in this paper, and verify in Section 4.1 that our choice of decoding strategy does not significantly impact our results.

While prior work proposed other definitions, we prefer ours in this paper as it is more actionable. Some memorization definitions, including lower-bounds on differential privacy (Dwork et al., 2006; Jagielski et al., 2020; Nasr et al., 2021) or counterfactual memorization (Feldman and Zhang, 2020; Zhang et al., 2021), require training hundreds or thousands of models, which is impractical for large language models. Alternatively, computing *exposure* (Carlini et al., 2019) requires thousands of generations per sequence, and is only designed for carefully crafted training examples. Finally, k -eidetic memorization (Carlini et al., 2020), is a useful definition for *unprompted* memorization, but less useful for tightly bounding memorization by prompting with training data (as we will do). Future work might explore how our three scaling observations apply to other definitions of memorization.

3.2 SELECTION OF EVALUATION DATA

Having chosen a definition, we next describe our evaluation procedure. Ideally, we would consider every sequence $x = [p \parallel s]$ in the model’s training dataset (where x has been split into a length- k prefix p and a suffix s). For each sequence, we would report if the model exactly reproduces s when prompted with p , following Definition 3.1. Unfortunately, performing this test on every sequence in the training data would be prohibitively expensive. For example, the largest 6 billion parameter GPT-Neo model has a throughput of roughly one 100-token generation per second on a V100 GPU. Extrapolating to the 800GB training dataset, this would require over 30 GPU-years of compute.

Instead, we query on a smaller subset of the training data, that still produces statistically confident estimates. In this paper we randomly choose subsets of roughly 50,000 sequences, allowing us to efficiently run inference in just a few hours. The primary criteria when choosing a subset of the training data is to obtain a representative sample that allows us to draw meaningful conclusions from the data. We consider two approaches to constructing a subset of the data.

Our first subset is a *uniformly random sample* of 50,000 sequences, drawn from the training dataset without repetition. While a uniform sample is useful to estimate the absolute amount of memorization in a model, it is poorly suited for studying how memorization scales with data properties that are *not* uniformly represented in the training set. For example, prior work has identified that *data duplication* (i.e., how often the same sequence is repeated either exactly or approximately) is an important factor for memorization. Yet, because the frequency of training data duplication decays extremely quickly (Lee et al., 2021), a uniformly random sample of 50,000 sequences (accounting for $\leq 0.02\%$ of the dataset) is unlikely to contain *any* signal that would allow us to accurately measure the tail of this repeated data distribution. A similar concern arises for measuring how memorization scales with prompt length, since very long sentences account for only a small fraction of the training set.

Therefore, our second subset is a random sample *normalized* by both sequence lengths and duplication counts, which allows us to accurately measure memorization of large language models in the worst-case, on highly duplicated data with long prompts. For each sequence length $\ell \in \{50, 100, 150, \dots, 500\}$, and integer n , we select 1,000 sequences of length ℓ that are contained in the training dataset between $2^{n/4}$ and $2^{(n+1)/4}$ times. We do this until we reach an n for which 1,000 sequences are not available. This gives us 1,000 sequences that repeat between 6 and 8 times ($\approx 2^{11/4}$ and $\approx 2^{12/4}$) and also 1,000 sequences that repeat between 724 and 861 times ($\approx 2^{38/4}$ and $\approx 2^{39/4}$). This biased sampling allows us to more accurately measure memorization as a function of a sample’s duplication factor and prompt length, without querying the entire dataset. Note that constructing this duplicate-normalized data subset requires some work, as efficiently identifying duplicate substrings in an 800GB training dataset is computationally challenging. We make use of the suffix array construction from Lee et al. (2021) (see Appendix).

For each length from 50 to 500 tokens, we collect 50,000 examples duplicated varying numbers of times, totaling roughly 500,000 sequences. For each sequence of length ℓ , we prompt the model with